Chapter 11 Inheritance and Polymorphism

Objectives

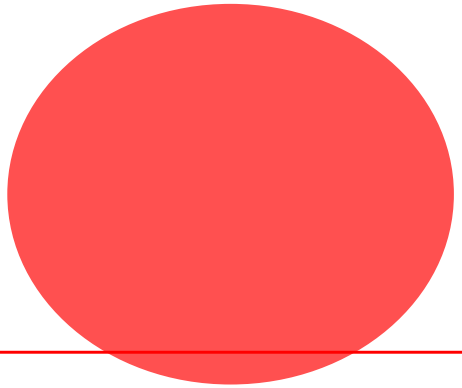
- To define a subclass from a superclass through inheritance (§11.2).
- To invoke the superclass's constructors and methods using the **super** keyword (§11.3).
- To override instance methods in the subclass (§11.4).
- To distinguish differences between overriding and overloading (§11.5).
- To explore the **toString()** method in the **Object** class (§11.6).
- To enable data and methods in a superclass accessible from subclasses using the **protected** visibility modifier (§11.13).
- To prevent class extending and method overriding using the **final** modifier (§11.14).

Motivation

Suppose you will define classes to model circles, rectangles, and triangles. These classes have many common features. What is the best way to design these classes so to avoid redundancy?

The answer is to use inheritance.

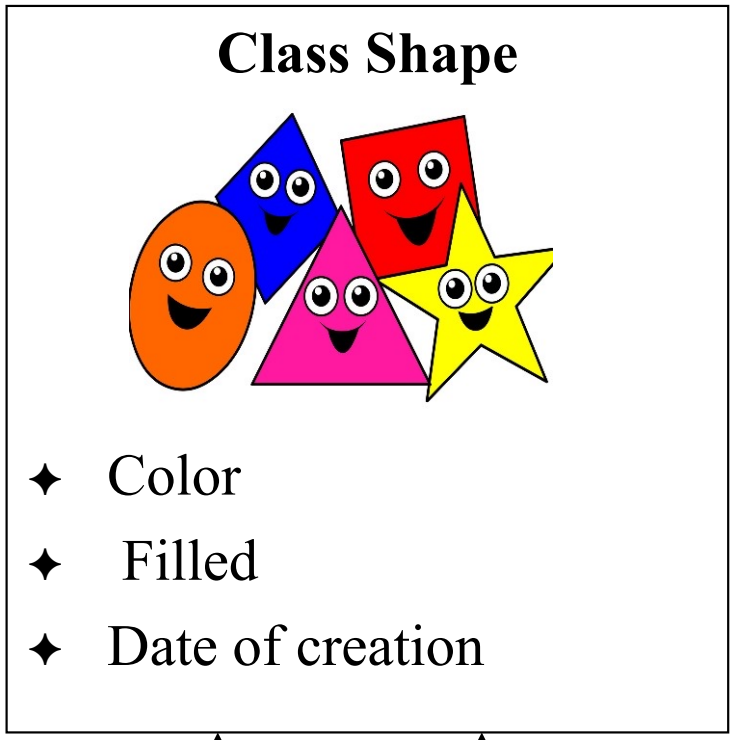
What are the attributes?



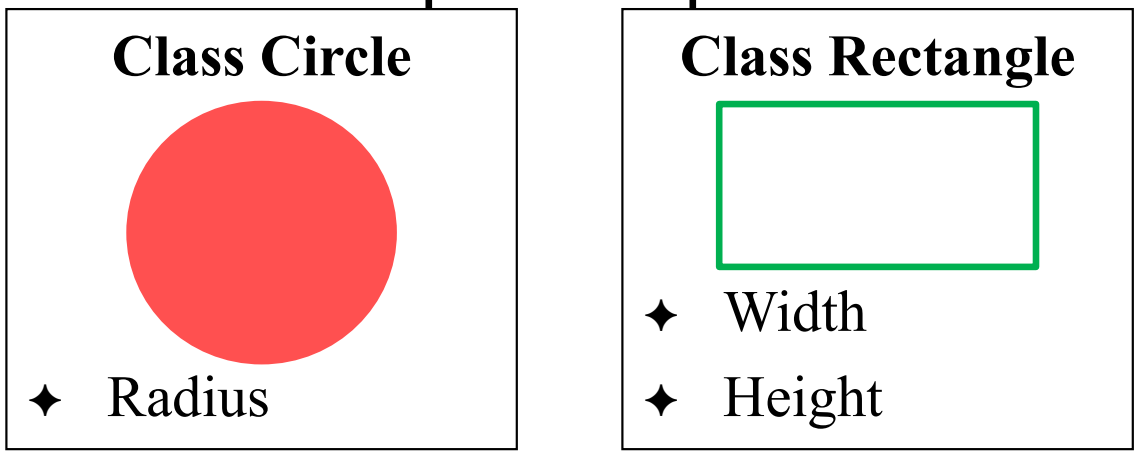
- Color = Red
- Filled = Yes
- Radius =3
- Date of creation

- Color =Green
- Filled = No
- Width = 4
- Height= 3
- Date of creation

What is common ?



SuperClass



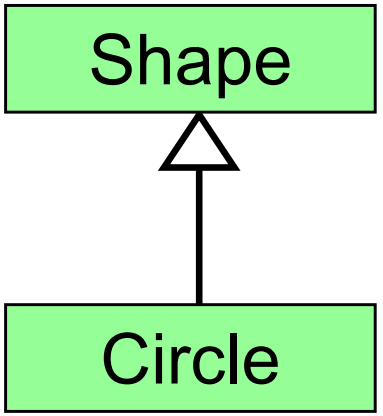
SubClasses

Inheritance

- *Inheritance* allows a software developer to derive a new class from an existing one.
- The existing class is called the *parent class*, or *superclass*, or *base class*.
- The derived class is called the *child class* or *subclass* or *extended class*.
- As the name implies, the child inherits characteristics of the parent.
- The child class inherits the methods and data defined by the parent class.

Inheritance

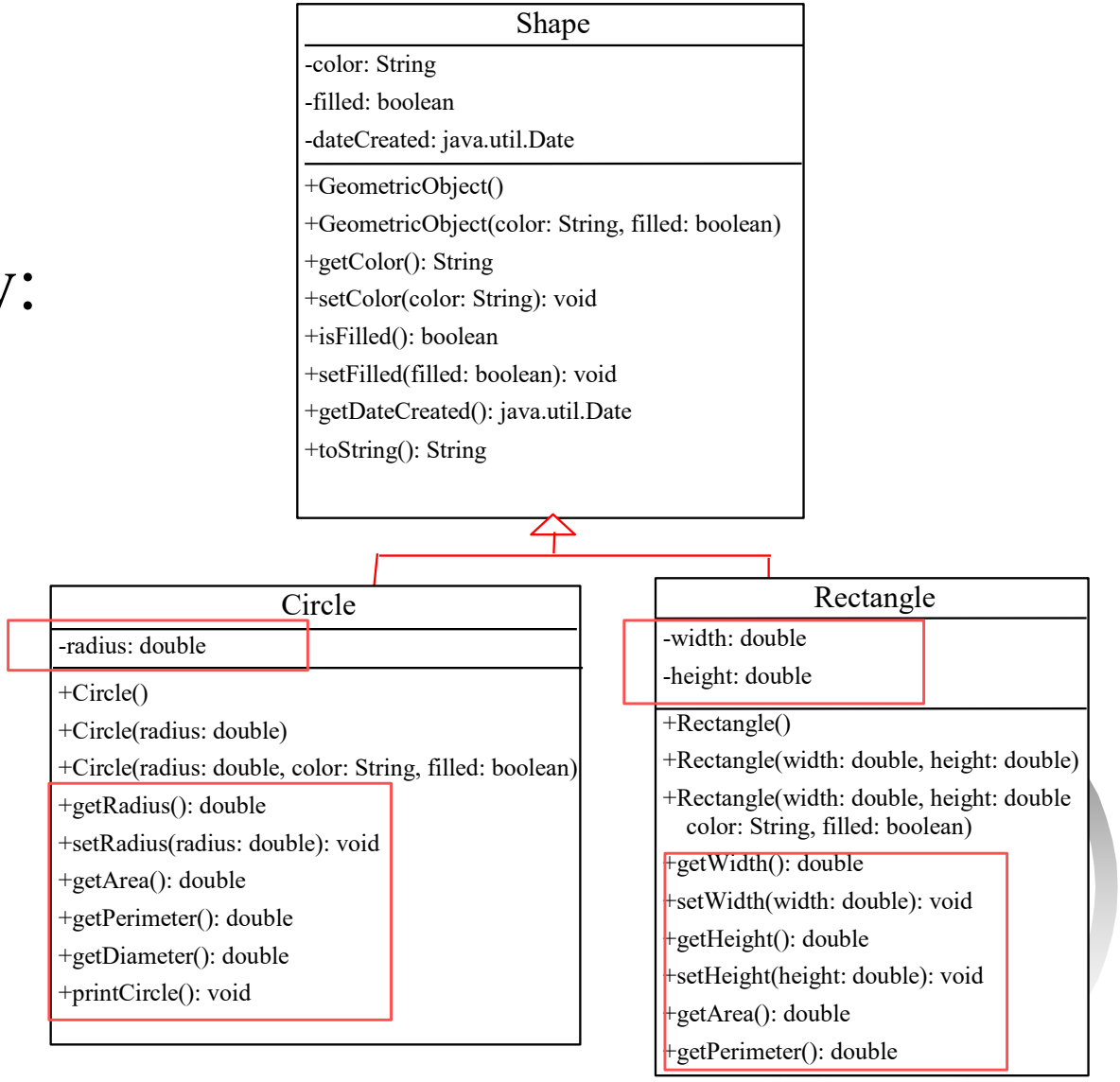
- Inheritance relationships are shown in a UML class diagram as a triangular arrow pointing to the superclass.



- Proper inheritance creates an **is-a** relationship, meaning the child is a more specific version of the parent.

Superclasses and Subclasses

- A programmer can tailor a child class as needed by adding new:
  - **variables or**
  - **methods, or**
  - **by modifying the inherited ones**



Inheritance

- *Software reuse* is a fundamental benefit of inheritance.
- By using existing software components to create new ones, we capitalize on all the effort that went into the design, implementation, and testing of the existing software.

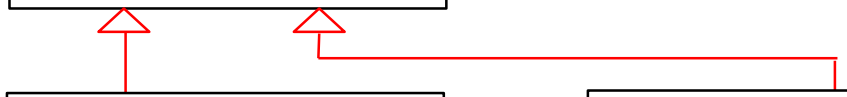
Defining a Subclass

A subclass inherits from a superclass, but we can also:

- Add new properties
- Add new methods
- Override the methods of the superclass

Superclasses and Subclasses

GeometricObject	
-color: String	The color of the object (default: white).
-filled: boolean	Indicates whether the object is filled with a color (default: false).
-dateCreated: java.util.Date	The date when the object was created.
+GeometricObject()	Creates a GeometricObject.
+GeometricObject(color: String, filled: boolean)	Creates a GeometricObject with the specified color and filled values.
+getColor(): String	Returns the color.
+setColor(color: String): void	Sets a new color.
+isFilled(): boolean	Returns the filled property.
+setFilled(filled: boolean): void	Sets a new filled property.
+getDateCreated(): java.util.Date	Returns the dateCreated.
+toString(): String	Returns a string representation of this object.



Circle
-radius: double
+Circle()
+Circle(radius: double)
+Circle(radius: double, color: String, filled: boolean)
+getRadius(): double
+setRadius(radius: double): void
+getArea(): double
+getPerimeter(): double
+getDiameter(): double
+printCircle(): void

Rectangle
-width: double
-height: double
+Rectangle()
+Rectangle(width: double, height: double)
+Rectangle(width: double, height: double color: String, filled: boolean)
+getWidth(): double
+setWidth(width: double): void
+getHeight(): double
+setHeight(height: double): void
+getArea(): double
+getPerimeter(): double

GeometricObject

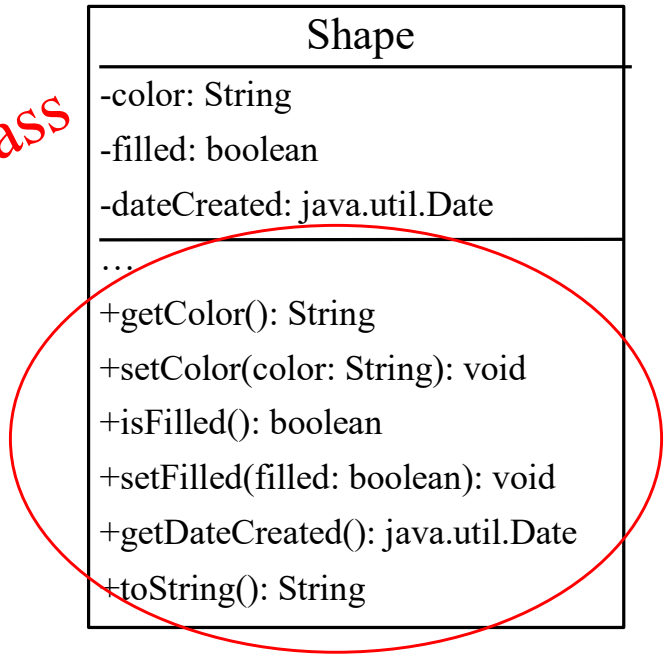
Circle

Rectangle

TestCircleRectangle

Run

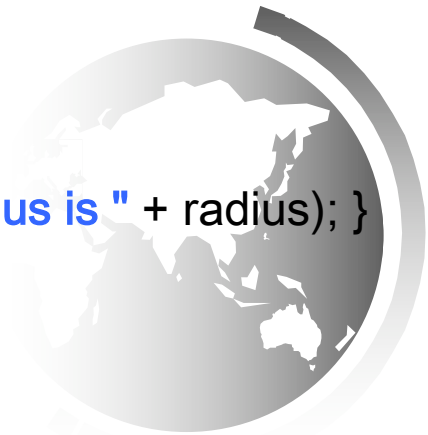
The Child's features



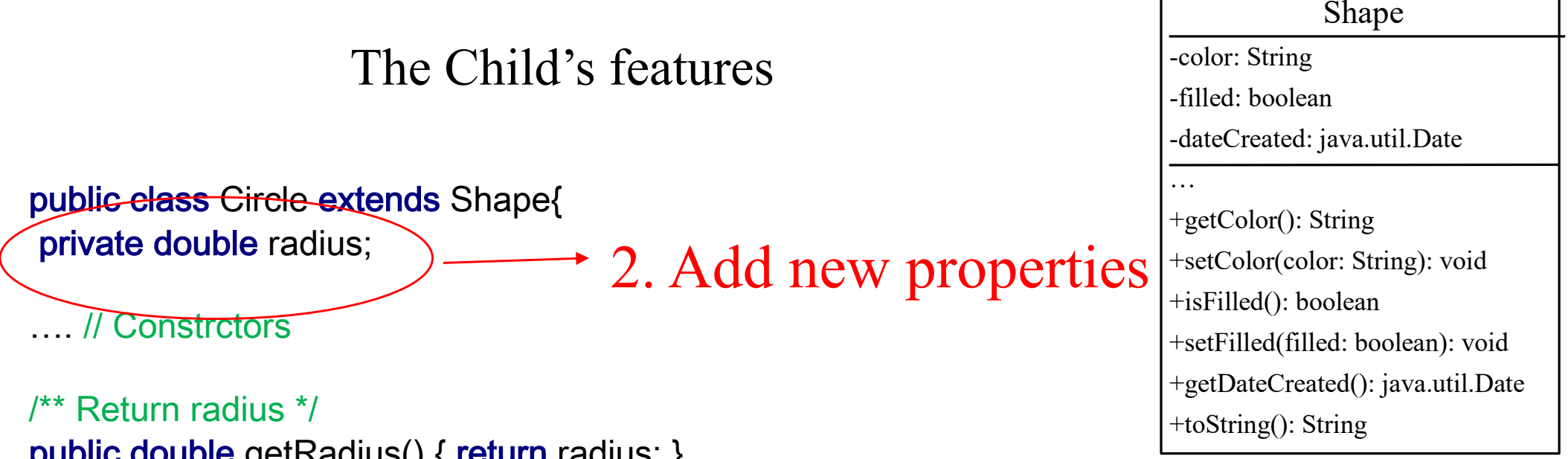
1. inherits from a superclass

```
public class Circle extends Shape{
 private double radius;

…. // Constrctors

/** Return radius */
public double getRadius() { return radius; }
 /** Set a new radius */
public void setRadius(double radius) { this.radius = radius; }
/** Return area */
public double getArea() { return radius * radius * Math.PI; }
/** Return diameter */
public double getDiameter() { return 2 * radius; }
/** Return perimeter */
public double getPerimeter() { return 2 * radius * Math.PI; }
 /* Print the circle info */
public void printCircle() {
System.out.println("The circle is created " + getDateCreated() + " and the radius is " + radius); }
}
```

Private data fields in a superclass are not accessible outside the class. Therefore, they cannot be used directly in a subclass.

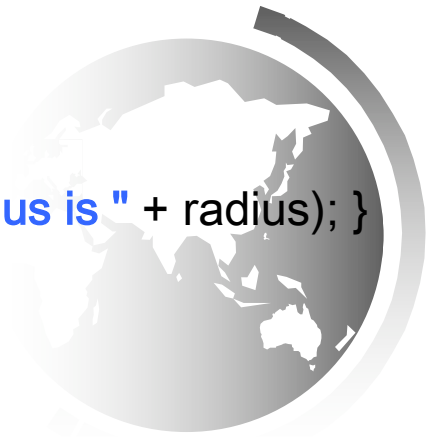


The Child's features




```
public class Circle extends Shape{
 private double radius;

…. // Constrctors

/** Return radius */
public double getRadius() { return radius; }
 /** Set a new radius */
public void setRadius(double radius) { this.radius = radius; }
/** Return area */
public double getArea() { return radius * radius * Math.PI; }
/** Return diameter */
public double getDiameter() { return 2 * radius; }
/** Return perimeter */
public double getPerimeter() { return 2 * radius * Math.PI; }
 /* Print the circle info */
public void printCircle() {
System.out.println("The circle is created " + getDateCreated() + " and the radius is " + radius); }
}
```

2. Add new properties

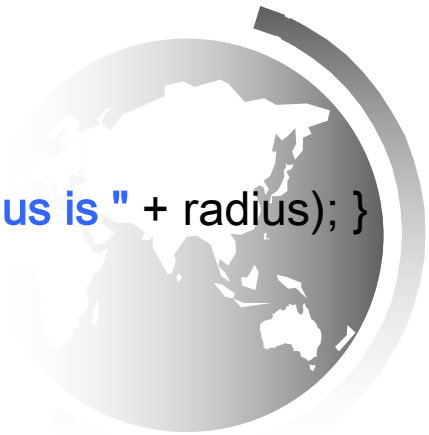


The Child's features

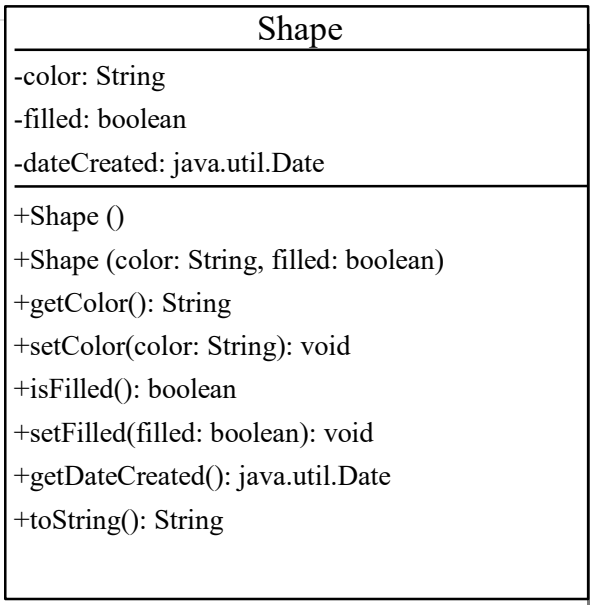
Shape
-color: String -filled: boolean -dateCreated: java.util.Date
… +getColor(): String +setColor(color: String): void +isFilled(): boolean +setFilled(filled: boolean): void +getDateCreated(): java.util.Date +toString(): String

3. Add new methods

```
public class Circle extends Shape{
 private double radius;

…. // Constrctors

/** Return radius */
public double getRadius() { return radius; }
 /** Set a new radius */
public void setRadius(double radius) { this.radius = radius; }
/** Return area */
public double getArea() { return radius * radius * Math.PI; }
/** Return diameter */
public double getDiameter() { return 2 * radius; }
/** Return perimeter */
public double getPerimeter() { return 2 * radius * Math.PI; }
 /* Print the circle info */
public void printCircle() {
System.out.println("The circle is created " + getDateCreated() + " and the radius is " + radius); }
}
```



```
public class Shape {
private String color = "white";
private boolean filled;
private java.util.Date dateCreated;

/** Construct a default object */
public Shape ()
{              dateCreated = new java.util.Date();         }
 /** Construct an object with the specified attributes
public Shape ( String color, boolean filled)
 {             dateCreated = new java.util.Date();         this.color = color;    this.filled = filled; }

/** Get and Set color */
public String getColor() { return color; }
 public void setColor(String color) { this.color = color; }

/** Get and Set filled. Since filled is boolean, the get is named isFilled */
public boolean isFilled() { return filled; }
public void setFilled(boolean filled) { this.filled = filled; }

/** Get dateCreated */
public java.util.Date getDateCreated() { return dateCreated; }

/** Return a string representation of this object */
public String toString()
{ return "created on " + dateCreated + "\ncolor: " + color + " and filled: " + filled; }
}
```



Extends reserved word

```
public class Circle extends Shape{

private double radius;

public Circle() {

  }
public Circle(double radius) {

  this.radius = radius;
}
public Circle(double radius, String color, boolean filled)
{
    this.radius = radius;
    setColor(color);
    setFilled(filled);
}

……

}
```

In Java, we use the reserved word `extends` to establish an inheritance relationship

this.color = color // Illegal

this.filled = filled // Illegal

Using the Keyword `super`

The keyword `super` refers to the superclass of the class in which `super` appears. This keyword can be used in two ways:

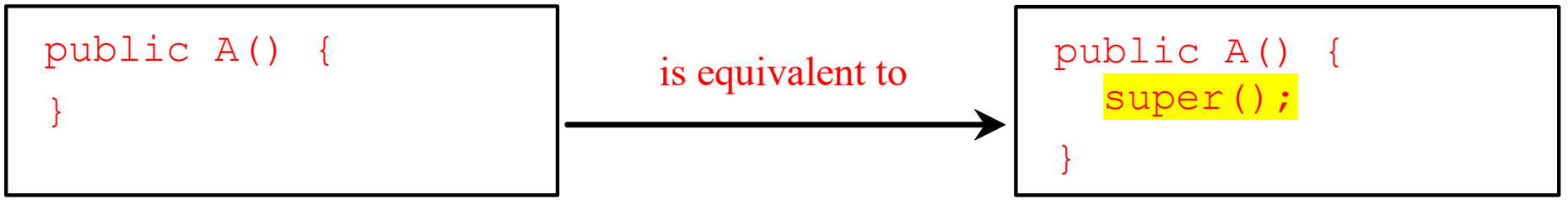
- To call a superclass constructor
- To call a superclass method

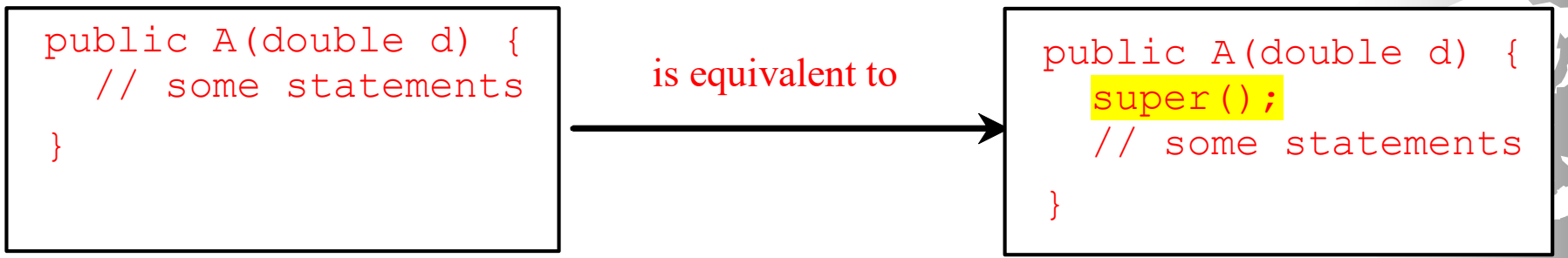
The super Reference

- A constructor is used to construct an instance of a class.
- Unlike properties and methods, a superclass's constructors are not inherited in the subclass.
- A child's constructor is responsible for calling the parent's constructor.
- They can only be invoked from the subclasses' constructors, using the keyword **super**.
- If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked.

Superclass's Constructor

A constructor may invoke an overloaded constructor (using this) or its superclass's constructor. If none of them is invoked explicitly, the compiler puts super() as the first statement in the constructor. For example,





Is the code shown below correct?

```
public class Shape {
public Shape () {… }
public Shape (String color, boolean
                    filled) {…..}
}//**************************************
public class Circle extends Shape{
public Circle() { }
}
```

√

```
public Circle() {
                }
```

is equivalent to

```
public Circle () {
   super();
}
```

// *invisible* call to superclass constructor

If the keyword **super** is not explicitly used, the superclass's no-arg constructor is automatically invoked.

Is the code shown below correct?

```
public class Shape {
public Shape () {… }
public Shape (String color, boolean
                    filled) {…..}
}//*************************************
public class Circle extends Shape{
   public Circle(double radius) {
       this.radius = radius;}}
```

√

```
public Circle(double radius){
     this.radius = radius;
}
```

is equivalent to

```
public Circle(double radius){
   super();
   this.radius = radius;
}
```

// *invisible* call to superclass constructor

Is the code shown below, correct?

```
public class Shape {
public Shape (String color, boolean
                    filled) {…..}
}//*************************************
public class Circle extends Shape{
public Circle() {   }
}
```

X

If no such call exists, Java will automatically make a call to super() at the beginning of the constructor, if the constructor is empty.

Is the code shown below correct?

```
public class Shape {

}//*************************************
public class Circle extends Shape{
public Circle() {   }
}
```

√

Java will automatically make a call to default super() at the beginning of the constructor, and the default constructor will be invoked since there aren't any constructors in the parent class.

Is the code shown below correct?

```
public class Shape {
public Shape () {… }
public Shape (String color, boolean
                    filled) {…..}
}//*************************************
public class Circle extends Shape{
public Circle() { super (“Red”,true) }
}
```

√

The first line of a child’s constructor used the `super` reference to call the parent’s constructor

Is the code shown below correct?

```
public class Shape {
public Shape () {… }
public Shape (String color, boolean
                    filled) {…..}
}//**************************************
public class Circle extends Shape{
}
```

√

Java will automatically make a call to super() at the beginning of the constructor, if there exist an empty constructor.

Example on the Impact of a Superclass without no-arg Constructor

Find out the errors in the program:

```
public class Apple extends Fruit {
}

class Fruit {
  public Fruit(String name) {
    System.out.println("Fruit's constructor is invoked");
  }
}
```

CAUTION

- You use the keyword super to call the superclass constructor.
- Invoking a superclass constructor's name in a subclass causes a syntax error.
- Java requires that the statement that uses the keyword super appear first in the constructor.

CAUTION

- Since the call to another constructor must be the *very first thing you do* in the constructor,
  - You can use this(...) to call another constructor in the same class:

```
class Foo extends Bar {
    Foo(String message) { // constructor
        this(message, 0, 0); // your explicit call to another constructor
        ...
```

  - You can use super(...) to call a specific *superclass* constructor

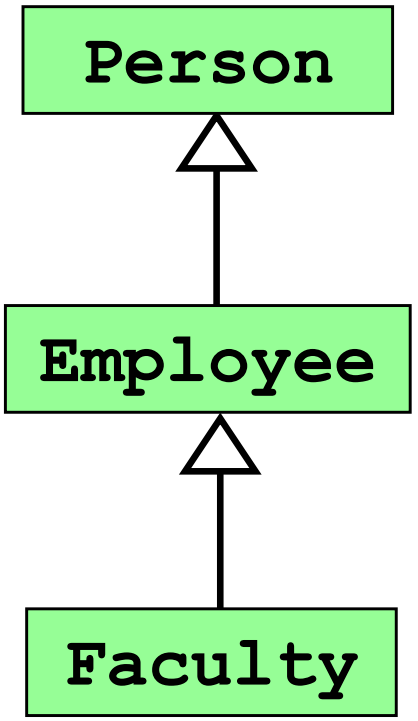
```
class Foo extends Bar {
    Foo(String name) { // constructor
        super(name, 5); // your explicit call to some superclass constructor
        ...
```

Constructors Cases

- No constructors → invoke Super Constructor
- No "this" inside another constructors → invoke Super
- "super" → invoke Super
- "this" constructor → invoke current constructor
- Command before "this" → error
- Command before "super" → error
- "super" with "this" → error

Constructor Chaining

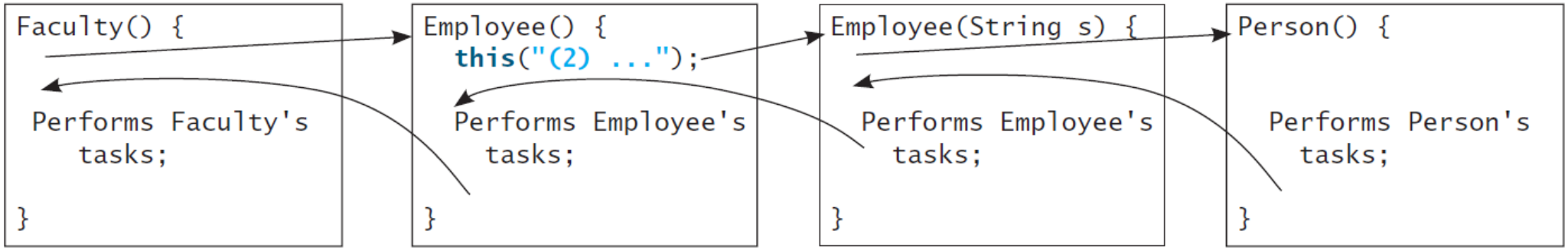
Constructing an instance of a class invokes all the superclasses' constructors along the inheritance chain. This is known as *constructor chaining*.



Constructor Chaining

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

Constructor Chaining



animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

1. Start from the main method

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

2. Invoke Faculty constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

3. Invoke Employee's no-arg constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

4. Invoke Employee(String) constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

5. Invoke Person() constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

6. Execute println

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

7. Execute println

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

8. Execute println

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

9. Execute println

Show the output of following program:

```
 1  public class Test {
 2    public static void main(String[] args) {
 3      A a = new A(3);
 4    }
 5  }
 6
 7  class A extends B {
 8    public A(int t) {
 9      System.out.println("A's constructor is invoked");
10    }
11  }
12
13  class B {
14    public B() {
15      System.out.println("B's constructor is invoked");
16    }
17  }
```

Show the output of following program:

```
 1  public class Test {
 2    public static void main(String[] args) {
 3      A a = new A(3);
 4    }
 5  }
 6
 7  class A extends B {
 8    public A(int t) {
 9      System.out.println("A's constructor is invoked");
10    }
11  }
12
13  class B {
14    public B() {
15      System.out.println("B's constructor is invoked");
16    }
17  }
```

B's constructor is invoked
A's constructor is invoked

Check Point

What is the output of running the class `C` in (a)? What problem arises in compiling the program in (b)?

```
class A {
  public A() {
    System.out.println(
      "A's no-arg constructor is invoked");
  }
}

class B extends A {
}

public class C {
  public static void main(String[] args) {
    B b = new B();
  }
}
```

(a)

```
class A {
  public A(int x) {
  }
}

class B extends A {
  public B() {
  }
}

public class C {
  public static void main(String[] args) {
    B b = new B();
  }
}
```

(b)

Calling Superclass Methods

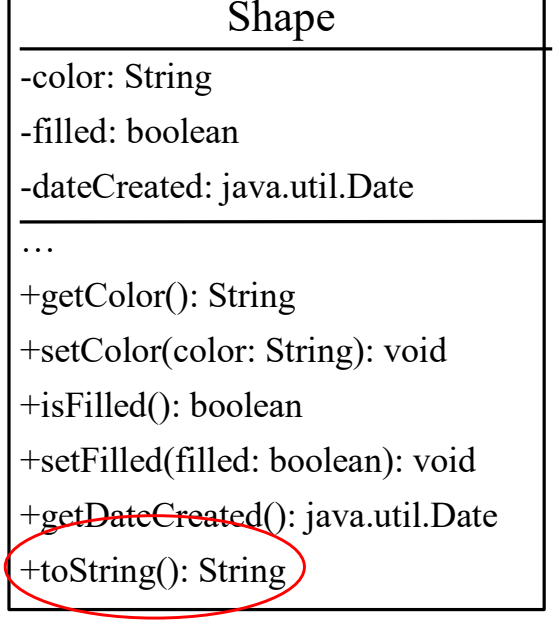
You could write the printCircle() method in the Circle class as follows:

```
public void printCircle() {
System.out.println("The circle is created " + getDateCreated()
+ " and the radius is " + radius); }
```

OR:

```
public void printCircle() {
  System.out.println("The circle is created " +
    super.getDateCreated() + " and the radius is " + radius);
}
```

The Child's features



4. Override the methods of the superclass

```
public class Circle extends Shape{
 private double radius;

…. // Constructors


/** Override the toString method defined in Shape class*/
public String toString() { return "The circle is created " + getDateCreated() + " and the radius is " +
                                  radius }
 ……


}
```

Overriding Methods in the Superclass

A subclass inherits methods from a superclass. Sometimes it is necessary for the subclass to modify the implementation of a method defined in the superclass. This is referred to as *method overriding*.

To override a method, the method must be defined in the subclass using the same signature and the same return type as in its superclass.

```
public class Circle extends Shape {
  // Other methods are omitted

  /** Override the toString method defined in Shape */
  public String toString() {
    return super.toString() + "\nradius is " + radius;
  }
}
```

NOTE

- An instance method can be overridden **only if it is accessible**.
- Thus, a private method cannot be overridden, because it is not accessible outside its own class.
- If a method defined in a subclass, is private in its superclass, then the two methods are completely unrelated.

What is the Output

```
class Base {
  private void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  private void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
     Derived obj = new Derived();
     obj.fun();
  }
}
```

Answer

```
class Base {
  private void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  private void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
     Derived obj = new Derived();
     obj.fun();
  }
}
```

```
Derived fun
```

What is the Output

```
class Base {
  private void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  public void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
     Derived obj = new Derived();
     obj.fun();
  }
}
```

Answer

```
class Base {
  private void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  public void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
     Derived obj = new Derived();
     obj.fun();
  }
}
```

```
Derived fun
```

Question & Answer

```
class Base {
  public void fun() {
     System.out.println("Base fun");
  }
}
public class Derived extends Base {
  private void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
      Derived obj = new Derived();
      obj.fun();
  }
}
```

error: fun() in Derived cannot override fun() in Base, attempting to assign weaker access privileges;

NOTE

- Like an instance method, a static method can be inherited. However, **a static method cannot be overridden**.
- If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden.
- The hidden static methods can be invoked using the syntax **SuperClassName.staticMethodName**.

What is the Output

```
class Base {
  public static void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  public static void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
      Derived obj = new Derived();
      obj.fun();
      Base.fun();
  }
}
```

Answer

```
class Base {
  public static void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  public static void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
      Derived obj = new Derived();
      obj.fun();
      Base.fun();
  }
}
```

```
Derived fun
Base fun
```

Question

```
class Base {
  public static void fun() {
     System.out.println("Base fun");
  }
}
class Derived extends Base {
      Derived(){
        super.fun();
      }
  public static void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
      Derived obj = new Derived();
      obj.fun();
      Base.fun();
  }
}
```

Answer

```
class Base {
  public static void fun() {
     System.out.println("Base fun");
  }
}
class Derived extends Base {
      Derived(){
        super.fun();
      }
  public static void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
      Derived obj = new Derived();
      obj.fun();
      Base.fun();
  }
}
```

```
Base fun
Derived fun
Base fun
```

Overriding vs. Overloading

- Overloading means to define multiple methods with the same name but different signatures. Overriding means to provide a new implementation for a method in the subclass.
- Overridden methods are in different classes related by inheritance; overloaded methods can be either in the same class or different classes related by inheritance.
- Overridden methods have the same signature and return type; overloaded methods have the same name but a different parameter list.

Overriding vs. Overloading

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

Overriding vs. Overloading

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

10.0
10.0

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

10
20.0

```
class A{

    void set (String v)
    {
        System.out.println("Hello A");
    }
}

class B extends A {

    void set (int h)
    {
        System.out.println("Hello B");
    }
}
public class Main
{
            public static void main(String[] args) {
                B b = new B();
                b.set(8);

            }
}
```

Overloading

```
class A{

   void set(String v)
   {
      System.out.println("Hello A");
   }
}

class B extends A {

   void set(int h)
   {
      System.out.println("Hello B");
   }
}
public class Main
{
         public static void main(String[] args) {
            B b = new B();
            b.set(8);

         }
}
```

Hello B

```
class A{

    void set (String v)
    {
        System.out.println("Hello A");
    }
}

class B extends A {

    void set (int h)
    {
        System.out.println("Hello B");
    }
}
public class Main
{
            public static void main(String[] args) {

                B b = new B();
                b.set("A");

            }
}
```

```
class A{

    void set(String v)
    {
        System.out.println("Hello A");
    }
}

class B extends A {

    void set(int h)
    {
        System.out.println("Hello B");
    }
}
public class Main
{
            public static void main(String[] args) {

                B b = new B();
                b.set("A");

            }
}
```

Overloading

Hello A

```
class A{

  void set (String v)
    {
      System.out.println("Hello A");
    }
}

class B extends A {


      void set (String h)
    {
       System.out.println("Hello B");
    }

}
public class Main
{
            public static void main(String[] args) {
               B b = new B();
               b.set("A");

            }
}
```

```
class A{

  void set(String v)
    {
      System.out.println("Hello A");
    }
}

class B extends A {


      void set(String h)
    {
       System.out.println("Hello B");
    }

}
public class Main
{
            public static void main(String[] args) {
               B b = new B();
               b.set("A");

            }
}
```

Overriding

Hello B

Override method: is the new version (behavior) of the old one. How can we reuse the old method plus adding the new behavior?

Calling Superclass Methods

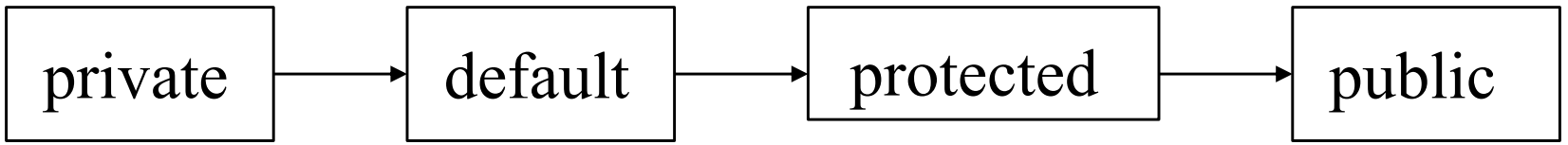
```
public class Shape {

……

public String toString() {
      return "created on " + dateCreated + "\ncolor: " + color + " and filled: " + filled; }
}
```

```
public class Circle extends Shape{
 private double radius;


public String toString() {
          return "The circle is created " + super.toString()+ " and the radius is " + radius }


}
```

The `protected` Modifier

- The `protected` modifier can be applied on data and methods in a class.
- A protected data or a protected method in a public class can be accessed by any class in:
  - the same package or
  - its subclasses, even if the subclasses are in a different package.

How the visibility increases?

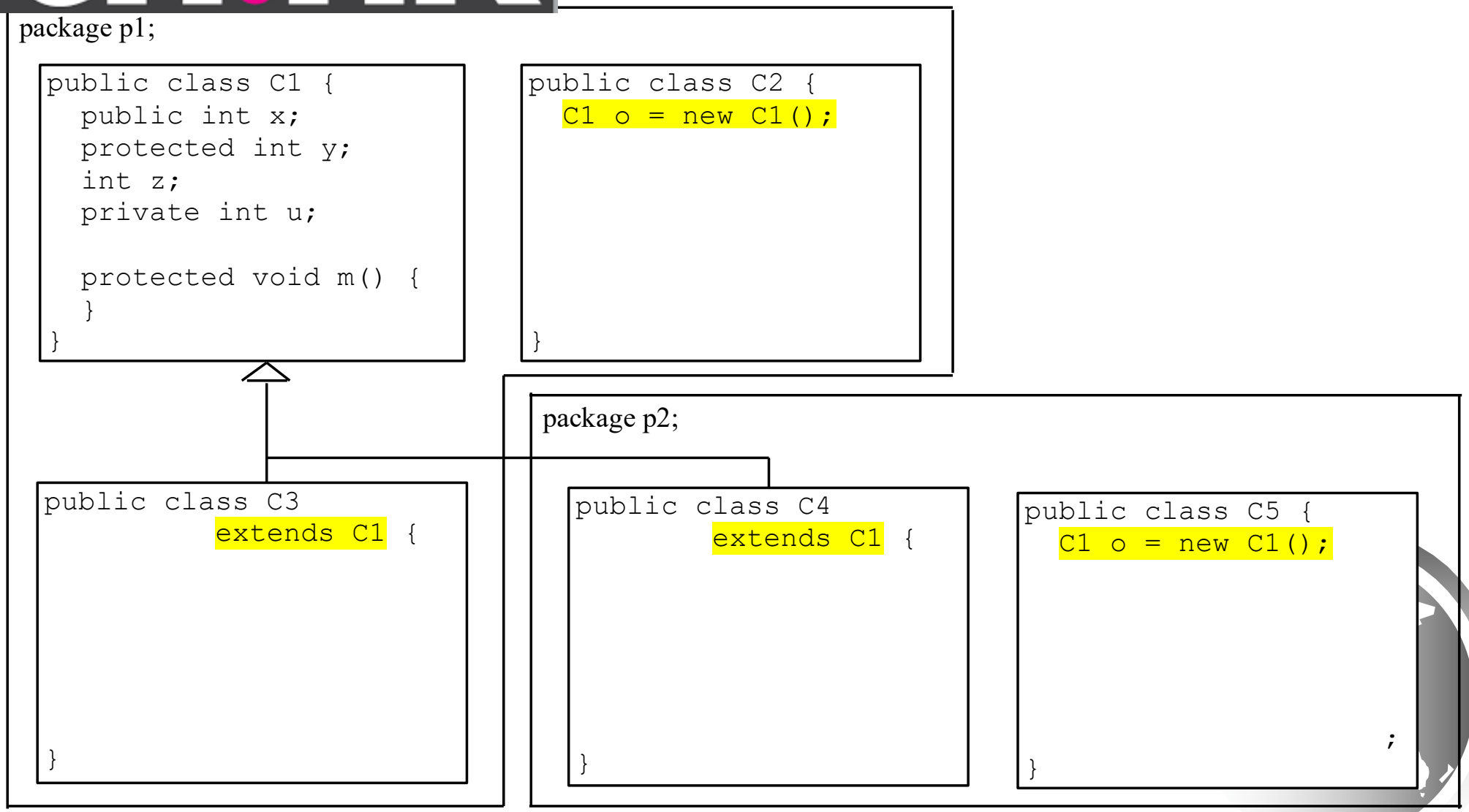


`none (if no modifier is used)`

Accessibility Summary

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass	Accessed from a different package
public				
protected				
default				
private				

Visibility Modifiers



Override the Accessibility

What is wrong with this code?

```
public class Shape {
……..

protected void Print()
{ // print the content }
…….
}
//**********************************
```

A Subclass cannot weaken the accessibility of a method defined in the superclass

```
public class Circle extends Shape{

….

private void Print () {/** Override the Print method defined in Shape class*/ }
 ……

}
```

Override the Accessibility

Is this code correct?

```
public class Shape {
……..

protected void Print()
{ // print the content }
…….
}
//**********************************
```

A subclass may override a protected method in its superclass and change its visibility to public

√

```
public class Circle extends Shape{

….

public void Print () {/** Override the Print method defined in Shape class*/ }
 ……

}
```

What is the output?

```
class Base {
  public void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  private void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
     Derived obj = new Derived();
     obj.fun();
  }
}
```

Answer

```
class Base {
  public void fun() {
     System.out.println("Base fun");
  }
}


class Derived extends Base {
  private void fun() {
    System.out.println("Derived fun");
  }
  public static void main(String[] args) {
     Derived obj = new Derived();
     obj.fun();
  }
}
```

Show the output of the following code:

```
public class Test {
  public static void main(String[] args) {
    new Person().printPerson();
    new Student().printPerson();
  }
}

class Student extends Person {
  @Override
  public String getInfo() {
    return "Student";
  }
}

class Person {
  public String getInfo() {
    return "Person";
  }

  public void printPerson() {
    System.out.println(getInfo());
  }
}
```

Show the output of the following code:

```
public class Test {
  public static void main(String[] args) {
    new Person().printPerson();
    new Student().printPerson();
  }
}

class Student extends Person {
  @Override
  public String getInfo() {
    return "Student";
  }
}

class Person {
  public String getInfo() {
    return "Person";
  }

  public void printPerson() {
    System.out.println(getInfo());
  }
}
```

Person
Student

Show the output of the following code:

```
public class Test {
  public static void main(String[] args) {
    new Person().printPerson();
    new Student().printPerson();
  }
}

class Student extends Person {
  private String getInfo() {
    return "Student";
  }
}

class Person {
  private String getInfo() {
    return "Person";
  }

  public void printPerson() {
    System.out.println(getInfo());
  }
}
```

Show the output of the following code:

```
public class Test {
  public static void main(String[] args) {
    new Person().printPerson();
    new Student().printPerson();
  }
}

class Student extends Person {
  private String getInfo() {
    return "Student";
  }
}

class Person {
  private String getInfo() {
    return "Person";
  }

  public void printPerson() {
    System.out.println(getInfo());
  }
}
```

Person
Person

Analyse the following code:

```
public class Test {
  public static void main(String[]
args) {
    new B();
 }

class A {
  int i = 7;
  public A() {
    System.out.println("i from A is "
+ i);}
  public void setI(int i) {
    this.i = 2 * i;
  }}

class B extends A {
  public B() {
    setI(20);
    // System.out.println("i from B
is " + i);
  }
  @Override
  public void setI(int i) {
    this.i = 3 * i;
  }}
```

A. The constructor of class A is not called.

B. The constructor of class A is called, and it displays "i from A is 7".

C. The constructor of class A is called, and it displays "i from A is 40".

D. The constructor of class A is called, and it displays "i from A is 60".

Analyze the following code:

```
public class Test {
  public static void main(String[]
args) {
    new B();
  }
class A {
  int i = 7;
  public A() {
    System.out.println("i from A is "
+ i);
  }
  public void setI(int i) {
    this.i = 2 * i;
  }}
class B extends A {
  public B() {
    setI(20);
    // System.out.println("i from B
is " + i);
  }
  @Override
  public void setI(int i) {
    this.i = 3 * i;
  }
}
```

A. The constructor of class A is not called.

B. The constructor of class A is called, and it displays "i from A is 7".

C. The constructor of class A is called, and it displays "i from A is 40".

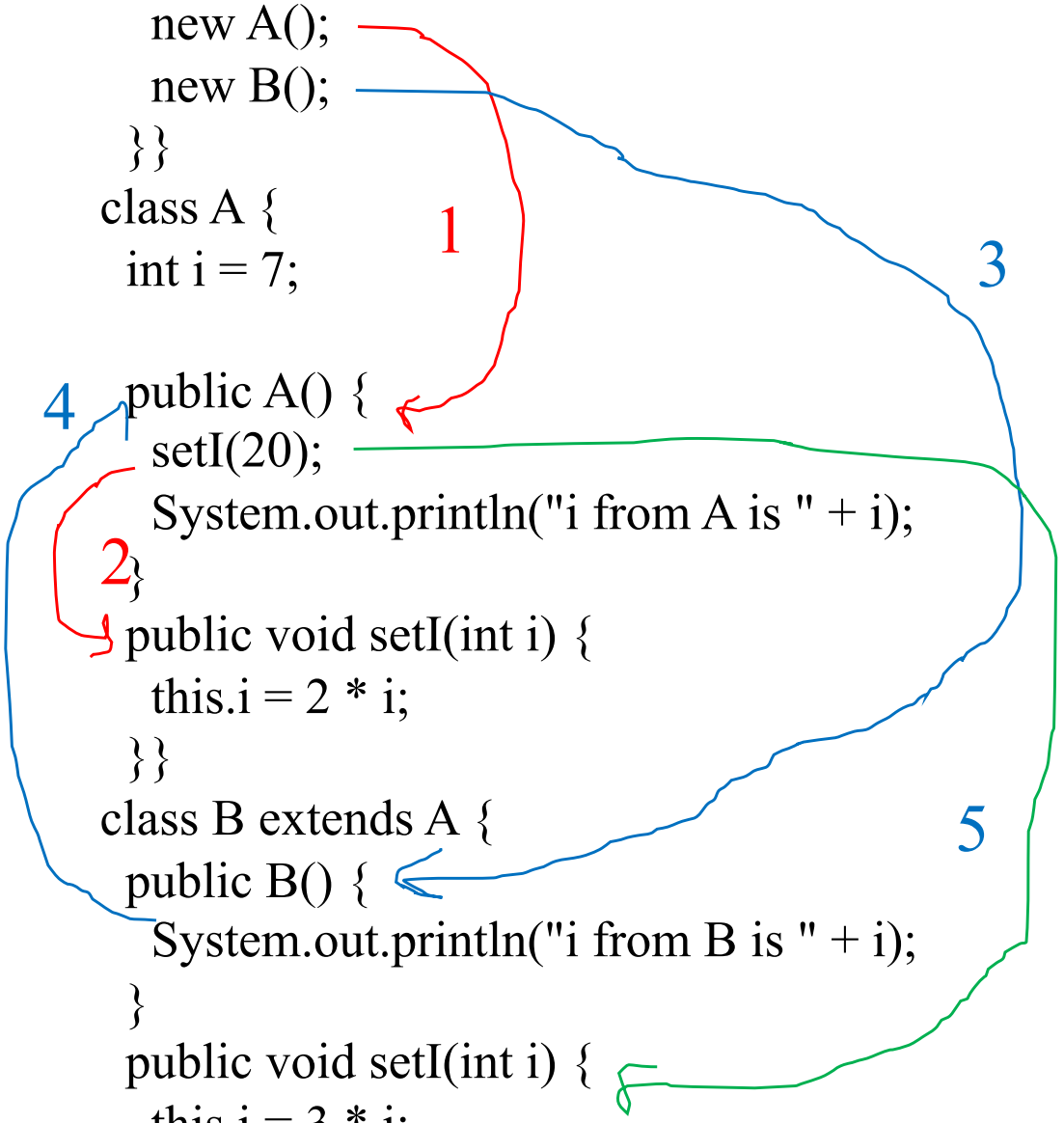
D. The constructor of class A is called, and it displays "i from A is 60".

Show the output of following program:

```
public class Test {
  public static void main(String[] args) {
    new A();
    new B();
  }}
class A {
  int i = 7;
  public A() {
    setI(20);
    System.out.println("i from A is " + i);
  }
  public void setI(int i) {
    this.i = 2 * i;
  }}
class B extends A {
  public B() {
    System.out.println("i from B is " + i);
  }
  public void setI(int i) {
    this.i = 3 * i;
  }
}
```

Show the output of following program:

```
public class Test {
  public static void main(String[] args) {
    new A();
    new B();
  }}
class A {
  int i = 7;

  public A() {
    setI(20);
    System.out.println("i from A is " + i);
  }
  public void setI(int i) {
    this.i = 2 * i;
  }}
class B extends A {
  public B() {
    System.out.println("i from B is " + i);
  }
  public void setI(int i) {
    this.i = 3 * i;
  }
}
```



i from A is 40
i from A is 60
i from B is 60

Analyse the following code:

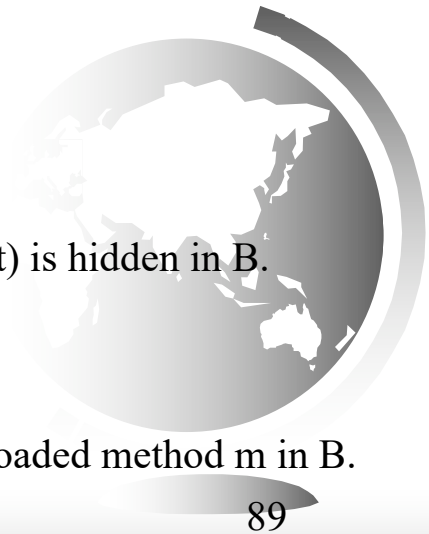
```
public class Test {
  public static void main(String[] args) {
    B b = new B();
    b.m(5);
    System.out.println("i is " + b.i);
  }
}
class A {
  int i;

  public void m(int i) {
    this.i = i;
  }
}

class B extends A {
  public void m(String s) {
  }
}
```

A. The program has a compile error, because m is overridden with a different signature in B.

B. The program has a compile error, because b.m(5) cannot be invoked since the method m(int) is hidden in B.

C. The program has a runtime error on b.i, because i is not accessible from b.

D. The method m is not overridden in B. B inherits the method m from A and defines an overloaded method m in B.



Analyse the following code:

```
public class Test {
  public static void main(String[] args) {
    B b = new B();
    b.m(5);
    System.out.println("i is " + b.i);
  }
}
class A {
  int i;

  public void m(int i) {
    this.i = i;
  }
}
class B extends A {
  public void m(String s) {
  }
}
```

A. The program has a compile error, because m is overridden with a different signature in B.

B. The program has a compile error, because b.m(5) cannot be invoked since the method m(int) is hidden in B.

C. The program has a runtime error on b.i, because i is not accessible from b.

D. The method m is not overridden in B. B inherits the method m from A and defines an overloaded method m in B.

The getValue() method is overridden in two ways. Which one is correct?

I:

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    System.out.println(a.getValue());
  }
}

class B {
  public String getValue() {
    return "Any object";
  }
}

class A extends B {
  public Object getValue() {
    return "A string";
  }
}
```

II:

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    System.out.println(a.getValue());
  }
}

class B {
  public Object getValue() {
    return "Any object";
  }
}

class A extends B {
  public String getValue() {
    return "A string";
  }
}
```

A. I

B. II

C. Both I and II

D. Neither

The getValue() method is overridden in two ways. Which one is correct?

I:

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    System.out.println(a.getValue());
  }
}

class B {
  public String getValue() {
    return "Any object";
  }
}

class A extends B {
  public Object getValue() {
    return "A string";
  }
}
```

II:

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    System.out.println(a.getValue());
  }
}

class B {
  public Object getValue() {
    return "Any object";
  }
}

class A extends B {
  public String getValue() {
    return "A string";
  }
}
```

A. I

B. II

C. Both I and II

D. Neither

The `final` Modifier

- The `final` variable is a constant:

```
final static double PI = 3.14159;
```

- The `final` class ?????

```
final class Math {
  ...
}
```

- The `final` method ?????

`final` class example

```
final class A {

// methods and fields
}

// The following class is illegal.
class B extends A {

// COMPILE-ERROR! Can't extends A

}
```

`Final` method example

```
class A {
    final void m1() {
            System.out.println("This is a final method."); }
}

class B extends A {
    void m1() {
   // COMPILE-ERROR! Can't override.
            System.out.println("Illegal!");
     }
 }
```

The Object Class
(The Grand parent of all classes)

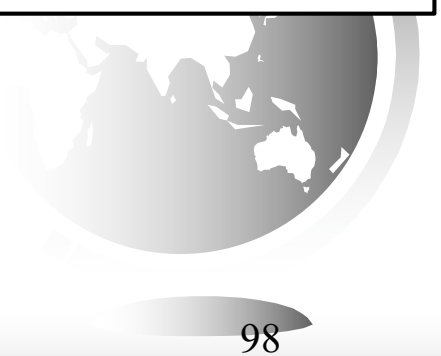
The Object Class and Its Methods

Every class in Java is descended from the java.lang.Object class. If no inheritance is specified when a class is defined, the superclass of the class is Object.

```
public class Circle {
  ...
}
```

Equivalent

```
public class Circle extends Object {
  ...
}
```



The toString() method in Object

The toString() method returns a string representation of the object.

The default implementation **(which is in the Object class)** returns a string consisting of:

```
Loan loan = new Loan();
System.out.println(loan.toString());
```

Or → `System.out.println(loan);`

Loan@15037e5

- a class name of which the object is an instance
- the at sign (@)
- a number representing this object (object's memory address in hexadecimal)

The toString()

Is this Object representation helpful?

Loan@15037e5

NO

SO, what do we do ?

Usually, you should override the toString method so that it returns a digestible string representation of the object. You can also pass an object to invoke System.out.print(object). This is equivalent to invoking System.out.print(object.toString()). Thus, you could replace System.out.println(loan.toString()) with System.out.println(loan).

Multiple Inheritance

- Java supports Only *single inheritance*, meaning that a derived class can have only one parent class.
- *Multiple inheritance* allows a class to be derived from two or more classes, inheriting the members of all parents (for example, pickup truck inherit from car and truck).
- Collisions, such as the same variable name in two parents, have to be resolved.
- Java does not support multiple inheritance.
- In most cases, the use of interfaces gives us aspects of multiple inheritance without the overhead.